

Discovery Executive Summary

Project: PingCRM-Discovery · Generated: 30/09/2026, 11:48:49

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 5 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—
2	Code Quality & Complexity Analysis	—
3	Testing & Quality Assurance Analysis	—
4	Performance & Sustainability Analysis	—
5	Technical Debt	—

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The PingCRM codebase is a Laravel 11 + React/TypeScript monolith comprising 89 PHP controllers (backend) and 769 TSX components (frontend). The architecture is severely compromised by a legacy IVR subsystem that was bolted onto the original clean CRM demo app. All 84 IVR controllers average 694 LOC each — over 4× the recommended ceiling — with raw SQL queries, `extract()` calls, and business logic embedded directly in HTTP handlers. Twelve “GodService” classes in `app/Legacy/Services/` each contain 324 LOC of duplicated `DB::table()` inserts with hard-coded secrets, and five legacy helper classes hold 567 LOC each of stub business logic. On the frontend, 124 legacy hooks perform inline `fetch()` calls with no shared data layer, and 133 `LegacyPass` placeholder components inflate the codebase without serving functional purpose. The dominant risk is **change amplification**: any schema or IVR business-rule change must be replicated across dozens of near-identical controller/service/repository files, making safe evolution practically impossible without first extracting bounded contexts and a service layer.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	636 LOC avg (84 IVR controllers at 694 each)	High Risk
H2	Missing Service Layer	Controllers accessing repos/models directly	<10	10–20	>20	87 controllers access models/DB directly	High Risk
H3	Missing Repository Pattern	Direct DB access points outside repositories	<10	10–20	>20	1,068 DB access points outside repos	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	Good
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	6	High Risk
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	~56% compliant	High Risk
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 files >1000 LOC; 12 GodService with 45 methods each	Moderate
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	3	Moderate
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	~15%	Moderate
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	144 avg LOC	Good

F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	124 legacy hooks with inline <code>fetch()</code>	High Risk
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	1 (Hub/Index.tsx at 479 LOC)	Moderate
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 levels	Good
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	362 (133 LegacyPass + 229 Monolith stubs)	High Risk

1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Fat Controllers	Collapse 55 legacy endpoints per controller into single parameterized method; move DB queries and business logic into services	High Risk	Critical
H2 — Missing Service Layer	Create Application Services per domain module; inject repository interfaces; move all business logic out of controllers	High Risk	Critical
H3 — Missing Repository Pattern	Replace 1,068 scattered DB access points with Eloquent repository implementations behind interfaces	High Risk	High
H5 — Shared Utility Abuse	Audit 5 legacy helper classes (2,835 LOC); delete dead code; consolidate survivors into domain-specific services	High Risk	Medium
H6 — Direct SQL in Controllers	Move all 111 raw SQL calls from controllers into repositories with parameterized bindings; eliminate SQL injection vectors	High Risk	Critical
H7 — God Classes	Consolidate 12 GodService × 45 methods into single parameterized orchestrate method; extract cache and config	Moderate	Medium
H8 — Domain Boundary Violations	Introduce OrganizationLookupInterface as anti-corruption layer between IVR and CRM domains	Moderate	Medium
H9 — Shared Database Coupling	Designate CRM as organizations table owner; IVR reads via published interface or read-only view	Moderate	Medium
F2 — Missing Frontend Service/Data Layer	Create shared API service (<code>api.ts</code>); replace 124 inline <code>fetch()</code> hooks with generic <code>useIvrModuleData()</code>	High Risk	High
F3 — God / Oversized Components	Split <code>Ivr/Hub/Index.tsx</code> (479 LOC) into sub-components: StatsCards, QueueTable, CallTable, AgentTable, DashboardFilters	Moderate	Low
F5 — Legacy / Inconsistent Component Patterns	Audit and delete 362 LegacyPass + Monolith placeholder stubs; migrate any survivors to Tailwind + hooks conventions	High Risk	High

1.5 Expected Outcomes

- **Separation of concerns:** Thin controllers delegate to Application Services, which orchestrate Domain Services and Repositories — each layer is independently testable and replaceable.
- **Elimination of SQL injection risk:** All raw SQL removed from controllers and replaced with parameterized repository queries, closing the most urgent security gap.
- **Independent domain evolution:** CRM and IVR bounded contexts communicate through published interfaces and an anti-corruption layer, enabling either domain to be extracted as a microservice without cross-team coordination on schema changes.
- **Drastically reduced change amplification:** Consolidating 4,620 duplicated legacy endpoint methods into parameterized services reduces the maintenance surface by ~99%, making IVR workflow changes safe single-point edits.
- **Clean frontend architecture:** A shared API service layer and removal of 362 dead placeholder files cut the frontend file count nearly in half while establishing a single, consistent convention for data fetching and component structure.
{"stop_reason":"","end_turn","session_id":"19b158e5-2f73-41a4-9de5-3315a8ba5849","total_cost_usd":3.0329945000000005,"usage":{"input_tokens":18,"cache_creation_input_tokens":161785,"cache_read_input_tokens":1185809,"output_tokens":32488,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":161785,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":{"input_tokens":1,"output_tokens":2351,"cache_read_input_tokens":111377,"cache_creation_input_tokens":11647,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":11647},"type":"message"},"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":9870,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.00995,"contextWindow":200000,"maxOutputTokens":32000,"opus-4-6":{"inputTokens":18,"outputTokens":32488,"cacheReadInputTokens":1185809,"cacheCreationInputTokens":161785,"webSearchRequests":0,"costUSD":3.0230445,"contextWindow":200000,"maxOutput":},"terminal_reason":"completed","fast_mode_state":"off","uuid":"00ab196b-c5e4-4d7a-ae83-c7ab108cae8d"}}

2. Code Quality & Complexity Analysis

EXECUTIVE SUMMARY

The PingCRM codebase suffers from extreme code duplication that dwarfs all other quality concerns. Approximately 70.5% of the 187,334 source lines are near-identical copies across 80 IVR controllers (759 LOC each, differing only by module name), 12 "GodService" classes, 8 legacyFormatters files, 133 LegacyPass2 React page components, and 147 legacy class widgets. The frontend layer (1,050 files / 107,920 LOC) is disproportionately affected, with duplicated utility files exceeding 1,000 LOC each and 133 single-function components at 387 LOC apiece. Backend duplication centres on a mechanically stamped controller-per-action pattern where every IVR module has 7 identical controllers. Cyclomatic complexity per method is moderate (max ~18), but

the sheer volume of copied code makes safe modification nearly impossible. Git churn is low (3 commits in 6 months), indicating the IVR layer was bulk-generated and has not yet been iterated on — the duplication debt will compound the moment feature work begins. Both layers were analysed: 180 PHP files (backend) and 1,050 TS/TSX/JS/JSS files (frontend).

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	~18 (Hub/Index.tsx component)	Moderate
H2	Large Classes	Largest class/file LOC	<300	300–1000	>1000	1,101 LOC (legacyFormatters*.ts — 8 files)	High Risk
H3	Large Functions	Largest function LOC	<50	50–200	>200	387 LOC (LegacyPass2 components — 133 files)	High Risk
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	~34% (GodServices + IVR controllers)	High Risk
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	~70.5% (132,077 of 187,334 LOC)	High Risk
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	<2/month (top file: composer.lock — 27 all-time)	Good
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	2 (HandleInertiaRequests.php, Users/Edit.vue)	Good
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	100% single-author (IVR layer); 52% top-author overall	Good
H9	Unsafe Dynamic Variables (additional)	Files using <code>extract()</code> on user input	0	1–5	>5	92 files (80 controllers + 12 GodServices)	High Risk

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	50	12.5
Code Churn	25%	10	2.5
Defect Density	20%	15	3.0
Class/Function Size	15%	75	11.25
Business Logic Duplication	10%	95	9.5
Developer Ownership Risk	5%	10	0.5
Hotspot Score	100%		39 / 100

2.5 Actions Required

Hotspot	Action	Rating	Priority
H2 — Large Classes	Replace 8 legacyFormatters files with single parameterised function; consolidate 80 IVR controllers into 1 generic controller; merge 12 GodServices into 1 service	High Risk	Critical
H3 — Large Functions	Replace 133 LegacyPass2 components with single data-driven component; split Hub/Index.tsx into sub-components	High Risk	Critical
H4 — Business Logic Duplication	Create module registry config; implement single IvModuleService with Command pattern for workflows	High Risk	Critical
H5 — Duplicate Code (general)	Consolidate all legacy frontend copies (formatters, Pass2 pages, class widgets, hooks) into parameterised originals	High Risk	Critical
H9 — Unsafe Dynamic Variables	Replace <code>extract(\$payload)</code> with explicit assignment; add Form Request validation; replace raw SQL with query builder	High Risk	High
H1 — High Cyclomatic Complexity	Decompose Hub/Index.tsx into sub-components; extract LoadIvrModuleData query patterns into service	Moderate	Medium

2.6 Expected Outcomes

- **~70% reduction in codebase size** — eliminating duplicated code reduces ~187,000 LOC to ~55,000 LOC, dramatically lowering maintenance burden and build times.
- **Safer modifications** — a single IvModuleService and IvModuleCrudController means business-rule changes are made once, not 92 times.

- **Improved static analysis** — removing `extract()` and raw SQL enables PHPStan/Psalm and ESLint to catch real bugs; currently these tools cannot trace variable origins.
 - **Faster onboarding** — new developers can understand 1 parameterised pattern instead of navigating 80 identical controller files.
 - **Smaller frontend bundle** — replacing 133 LegacyPass2 components and 147 class widgets with data-driven alternatives reduces JS bundle size by an estimated 60+ KB
- ```
(gzipipped)","stop_reason":"end_turn","session_id":"8d234490-a6c9-4803-97b6-a619500ff6c7","total_cost_usd":2.4659875,"usage":{"input_tokens":23,"cache_creation_input_tokens":97860,"cache_read_input_tokens":1283219,"output_tokens":33490,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":97860,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":{"input_tokens":1,"output_tokens":2221,"cache_read_input_tokens":87012,"cache_creation_input_tokens":9519,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":9519},"type":"message"},"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":8328,"outputTokens":17,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.008413,"contextWindow":200000,"maxOutputTokens":32000,"opus-4-6":{"inputTokens":23,"outputTokens":33490,"cacheReadInputTokens":1283219,"cacheCreationInputTokens":97860,"webSearchRequests":0,"costUSD":2.4575745,"contextWindow":200000,"maxOutputTokens":33490,"terminal_reason":"completed","fast_mode_state":"off","uuid":"4b09a394-70d8-46cb-b1e2-4035f1775235"}}
```

3. Testing & Quality Assurance Analysis

EXECUTIVE SUMMARY

Ping CRM is a Laravel 11 + React 19 (Inertia.js) application with a large IVR enterprise module surface. The test suite is critically thin: only 3 meaningful PHPUnit test files cover 2 of 141 backend source files (2.1% file coverage), and the frontend has a single assertion-less smoke test for 904 TypeScript/React source files (0.1%). Authentication, user management, the entire IVR enterprise module (91 controllers, 12 legacy "GodService" classes, 12 repositories), reports, and all 14 shared UI components ship with zero automated test coverage. The CI pipeline (`tests.yml`) runs PHPUnit on push/PR but does not run Vitest, so the frontend is never gate-checked. No integration tests exercise real database boundaries beyond the two existing feature tests, no contract tests exist for the IVR module's dynamic routing, and no end-to-end tests verify user journeys.

5.1 Benchmark Ratings Summary

| #  | Hotspot                           | Primary KPI                         | <span class="rating rating-good">Good | <span class="rating rating-moderate">Moderate | <span class="rating rating-high-risk">High Risk | Measured                                                                                                  | Rating                                          |
|----|-----------------------------------|-------------------------------------|---------------------------------------|-----------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| H1 | Untested Critical Logic           | Critical modules with zero tests    | 0                                     | 1–3                                           | >3                                              | 7 critical modules (auth, users, reports, IVR hub, IVR module controllers, legacy services, repositories) | <span class="rating rating-high-risk">High Risk |
| H2 | Low Test Coverage                 | Overall coverage %                  | >80%                                  | 50–80%                                        | <50%                                            | ~2% backend / ~0% frontend (estimated from test-file-to-source ratio)                                     | <span class="rating rating-high-risk">High Risk |
| H3 | Missing Integration Tests         | Boundaries covered %                | >70%                                  | 30–70%                                        | <30%                                            | ~5% — only ContactsTest and OrganizationsTest exercise DB                                                 | <span class="rating rating-high-risk">High Risk |
| H4 | Missing Contract Tests            | APIs with contract tests %          | >80%                                  | 40–80%                                        | <40%                                            | 0% — no contract or schema validation tests exist                                                         | <span class="rating rating-high-risk">High Risk |
| H5 | Flaky / Skipped Tests             | Skipped/flaky test count            | 0                                     | 1–5                                           | >5                                              | 0                                                                                                         | <span class="rating rating-good">Good           |
| H6 | No CI Test Gate                   | Tests enforced in CI                | Required gate                         | Runs, not required                            | No CI test run                                  | PHPUnit runs on push/PR (not required); Vitest never runs in CI                                           | <span class="rating rating-high-risk">High Risk |
| H7 | No End-to-End Tests (additional)  | E2E test suites present             | ≥1 suite                              | Partial/smoke only                            | None                                            | 0 — no Cypress, Playwright, or Dusk configured                                                            | <span class="rating rating-high-risk">High Risk |
| H8 | Assertion-Free Tests (additional) | Tests with no meaningful assertions | 0                                     | 1–2                                           | >2                                              | 2 — <code>ExampleTest.php</code> and <code>smoke.test.ts</code>                                           | <span class="rating rating-moderate">Moderate   |

5.4 Actions Required

| Hotspot                        | Action                                                                                                           | Rating                                          | Priority                                |
|--------------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------|
| H1 — Untested Critical Logic   | Generate unit and feature tests for auth, users, reports, IVR controllers, legacy services, and repositories     | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| H2 — Low Test Coverage         | Enable coverage reporting; target 50% backend and 30% frontend coverage in first sprint; set CI thresholds       | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| H3 — Missing Integration Tests | Add integration tests for ReportsController raw queries, IVR Store/Update/Destroy cycles, and auth session flow  | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High         |
| H4 — Missing Contract Tests    | Add Inertia assertInertia tests for all routes; add CSV output format tests; parameterized test for IVR SLUG_MAP | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High         |

|                           |                                                                                                  |                                                 |                                     |
|---------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------|-------------------------------------|
| H6 — No CI Test Gate      | Add Vitest to CI pipeline; configure branch protection requiring tests + static-analysis to pass | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High     |
| H7 — No End-to-End Tests  | Install Playwright or Dusk; write smoke E2E for login, IVR navigation, and CRUD flows; add to CI | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High     |
| H8 — Assertion-Free Tests | Replace ExampleTest.php and smoke.test.ts with real application assertions                       | <span class="rating rating-moderate">Moderate   | <span class="sev sev-medium">Medium |

5.5 Expected Outcomes

- **Critical auth and user-management paths protected** — login, logout, rate limiting, user CRUD, and demo-user guards verified automatically before every merge, preventing security regressions.
- **IVR enterprise module regression-safe** — the 91 IVR controllers and 12 legacy services gain integration test coverage, catching broken DB queries and data corruption before production.
- **CI catches regressions on both layers** — with Vitest added to the pipeline and branch protection enabled, no code (backend or frontend) merges without passing tests.
- **Contract tests prevent silent inertia breaks** — every route's prop structure is validated, eliminating the class of bugs where PHP changes break React rendering at runtime.
- **E2E tests verify real user journeys** — the login-to-IVR-module flow and CRUD operations are tested in a real browser, catching hydration failures and navigation bugs that unit tests cannot detect.  
{"stop\_reason":"","end\_turn","session\_id":"6a91ab74-397f-4455-adbf-213b2f5f78e9","total\_cost\_usd":1.9992385000000001,"usage":{"input\_tokens":21,"cache\_creation\_input\_tokens":84591,"cache\_read\_input\_tokens":1148683,"output\_tokens":22883,"server\_tool\_use":{"web\_search\_requests":0,"web\_fetch\_requests":0},"service\_tier":"standard","cache\_creation":{"ephemeral\_1h\_input\_tokens":84591,"ephemeral\_5m\_input\_tokens":0},"inference\_geo":"not\_available","iterations":{"input\_tokens":1,"output\_tokens":1713,"cache\_read\_input\_tokens":92916,"cache\_creation\_input\_tokens":481,"cache\_creation":{"ephemeral\_5m\_input\_tokens":0,"ephemeral\_1h\_input\_tokens":481},"type":"message"}},{"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":6727,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.0068070000000000006,"contextWindow":200000,"maxOutputOpus-4-6":{"inputTokens":21,"outputTokens":22883,"cacheReadInputTokens":1148683,"cacheCreationInputTokens":84591,"webSearchRequests":0,"costUSD":1.9924315000000001,"contextWindow":200000,"terminal\_reason":"completed","fast\_mode\_state":"off","uuid":"ae43520a-e665-467d-a389-1d2dff37e858"}}

4. Performance & Sustainability Analysis

EXECUTIVE SUMMARY

The PingCRM codebase has serious runtime performance problems concentrated in its IVR (Interactive Voice Response) legacy layer. Across 12 GodService classes, 540 synchronous `sleep(1)` calls block the PHP process for up to 55 seconds per request chain, making the legacy IVR endpoints effectively unusable at scale. In the frontend, 374 React page components create 5-second polling intervals via `setInterval` without cleanup (`clearInterval`), causing accumulated memory leaks and runaway network traffic as users navigate the SPA. Additionally, 80+ IVR controller endpoints load entire database tables into memory without pagination or `limit()` creating unbounded memory consumption. The well-written core CRM controllers (Contacts, Organizations) use proper pagination and query scoping, but the IVR legacy surface — which dominates the codebase at ~900+ files — undermines the application's overall performance posture. No infrastructure-as-code, autoscaling config, or resource sizing is present in the repository; Heroku deployment relies on a single Procfile with default dyno settings.

7.1 Benchmark Ratings Summary

| #  | Hotspot              | Primary KPI                                                       | <span class="rating rating-good">Good | <span class="rating rating-moderate">Moderate | <span class="rating rating-high-risk">High Risk | Measured                      | Rating                                          |
|----|----------------------|-------------------------------------------------------------------|---------------------------------------|-----------------------------------------------|-------------------------------------------------|-------------------------------|-------------------------------------------------|
| P1 | Algorithm Efficiency | High-complexity algorithm sites                                   | 0                                     | 1–5                                           | >5                                              | 0                             | <span class="rating rating-good">Good           |
| P2 | Database Performance | Deferred → Backend Modernization (H14/H10)                        | —                                     | —                                             | —                                               | See Backend Modernization     | — (deferred)                                    |
| P3 | API Performance      | Response-latency hotspots                                         | 0                                     | 1–5                                           | >5                                              | 540 (sleep-blocked endpoints) | <span class="rating rating-high-risk">High Risk |
| P4 | Memory Efficiency    | High-memory sites                                                 | 0                                     | 1–3                                           | >3                                              | 84 (unbounded get() calls)    | <span class="rating rating-high-risk">High Risk |
| P5 | CPU Efficiency       | CPU-intensive operations                                          | 0                                     | 1–5                                           | >5                                              | 1 (Glide image processing)    | <span class="rating rating-moderate">Moderate   |
| P6 | Concurrency          | Parallelizable work + pool sizing                                 | 0                                     | 1–5                                           | >5                                              | 2 (serial dashboard queries)  | <span class="rating rating-moderate">Moderate   |
| P7 | Caching              | Deferred → Backend Modernization H14 / Frontend Modernization H11 | —                                     | —                                             | —                                               | See those reports             | — (deferred)                                    |
| P8 | Resource Utilization | Over-provisioned / idle resources                                 | 0                                     | 1–3                                           | >3                                              | N/A — no infra config in repo | <span class="rating rating-good">Good           |

|     |                                    |                                                  |           |         |                   |                                                                     |                                                 |
|-----|------------------------------------|--------------------------------------------------|-----------|---------|-------------------|---------------------------------------------------------------------|-------------------------------------------------|
| P9  | Network Efficiency                 | Excessive-traffic sites                          | 0         | 1–5     | >5                | 498 (374 leaked polling intervals + 124 legacy fetch hooks)         | <span class="rating rating-high-risk">High Risk |
| P10 | Build Efficiency                   | Build/test pipeline efficiency                   | efficient | partial | slow / no caching | partial (composer cache present, npm cache absent)                  | <span class="rating rating-moderate">Moderate   |
| P11 | Logging Efficiency                 | Excessive-logging sites                          | 0         | 1–10    | >10               | 0                                                                   | <span class="rating rating-good">Good           |
| P12 | Sustainability                     | Resource-optimization posture                    | optimized | partial | wasteful          | wasteful (sleep-blocked processes, runaway polling, no autoscaling) | <span class="rating rating-high-risk">High Risk |
| P13 | Interval Memory Leaks (additional) | Components with setInterval and no clearInterval | 0         | 1–10    | >10               | 374                                                                 | <span class="rating rating-high-risk">High Risk |

7.5 Actions Required

| Hotspot                     | Action                                                                                                                                                      | Rating                                          | Priority                                |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------|
| P3 — API Performance        | Remove all 540 <code>sleep(1)</code> calls from GodServices; replace with queue jobs if async work is needed                                                | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| P4 — Memory Efficiency      | Add <code>paginate()</code> or <code>cursor()</code> to all 84 unbounded <code>-&gt;get()</code> calls in IVR controllers and core CRM (Users, Reports CSV) | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| P9 — Network Efficiency     | Add <code>clearInterval</code> cleanup to 374 IVR pages; add <code>AbortController</code> to 124 legacy hooks; consolidate into shared hooks                | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| P13 — Interval Memory Leaks | Fix 374 <code>useEffect</code> hooks missing cleanup returns; add ESLint rule to prevent recurrence                                                         | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| P12 — Sustainability        | Eliminate waste (sleep, leaked polls, unbounded queries); add Heroku autoscaling and resource limits                                                        | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High         |
| P5 — CPU Efficiency         | Pre-generate image variants at upload; serve cached images via web server                                                                                   | <span class="rating rating-moderate">Moderate   | <span class="sev sev-medium">Medium     |
| P6 — Concurrency            | Parallelize 7 independent dashboard queries using Laravel Concurrency or async                                                                              | <span class="rating rating-moderate">Moderate   | <span class="sev sev-medium">Medium     |
| P10 — Build Efficiency      | Add npm dependency caching to CI workflow; cache Vite build output                                                                                          | <span class="rating rating-moderate">Moderate   | <span class="sev sev-low">Low           |

7.6 Expected Outcomes

- **Removing `sleep(1)` calls** would reclaim ~540 seconds of blocked PHP worker time per full legacy endpoint sweep, increasing effective throughput by orders of magnitude on the IVR surface.
  - **Adding pagination** to unbounded queries would cap per-request memory from potentially 50–100 MB down to <5 MB regardless of data volume, eliminating OOM risk.
  - **Fixing interval leaks** would reduce unnecessary background network traffic by ~95%, cutting Heroku dyno load from thousands of phantom requests/minute to near zero for navigated-away pages.
  - **Parallelizing dashboard queries** could reduce IVR Hub page load time by 40–60%, improving the primary user experience for call center operators.
  - **Adding npm CI caching** would save 30–60 seconds per CI run, reducing developer feedback loops and CI compute costs.
- stop\_reasonend\_turnsession\_idd9e8cffc-01a0-4edd-b0eb-52e57c9e79be"total\_cost\_usd":3.2752334999999997,"usage":{"input\_tokens":35,"cache\_creation\_input\_tokens":126183,"cache\_read\_input\_tokens":2705243,"output\_tokens":26097,"server\_tool\_use":{"web\_search\_requests":0,"web\_fetch\_requests":0},"service\_tier":"standard","cache\_creation":{"ephemeral\_1h\_input\_tokens":126183,"ephemeral\_5m\_input\_tokens":0},"inference\_geo":"not\_available","iterations":{"input\_tokens":1,"output\_tokens":1954,"cache\_read\_input\_tokens":125361,"cache\_creation\_input\_tokens":8253,"cache\_creation":{"ephemeral\_5m\_input\_tokens":0,"ephemeral\_1h\_input\_tokens":8253},"type":"message"},"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":8102,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.008182,"contextWindow":200000,"maxOutputTokens":32000,"opus-4-6":{"inputTokens":35,"outputTokens":26097,"cacheReadInputTokens":2705243,"cacheCreationInputTokens":126183,"webSearchRequests":0,"costUSD":3.2670515,"contextWindow":200000,"maxOutput":},"terminal\_reason":"completed","fast\_mode\_state":"off","uuid":"f621cf56-d05e-404c-9654-9b698ae84ecc"}

5. Technical Debt

EXECUTIVE SUMMARY

PingCRM is a Laravel 11 / React 19 demo application with a deliberately injected legacy IVR monolith surface totalling ~185k lines across 800+ generated files. The core CRM is well-structured with CI pipelines for tests, coding standards, and static analysis; lock files are committed and `.env.example` is present. However, the legacy IVR layer introduces severe technical debt: hard-coded secrets committed in `config/ivr_legacy.php`, `extract()` calls in God services, raw SQL concatenation in 10+ repository files, zero foreign keys on the original CRM tables (`users`, `contacts`, `organizations`), and no Docker or devcontainer configuration for reproducible environments. There is no `CODEOWNERS` file, no branch protection signals, no pre-commit hooks enforcing linters, and no AI-assisted tooling config (`.cursor/`, `CLAUDE.md`, `.kiro/`). PHPStan is configured at only level 1 out of 9. The

codebase is **not ready** for an agentic harness today — the committed secrets alone block clean automation, and the absence of containerized environments makes reproducible agent-driven runs unreliable.

Readiness Benchmark Ratings

| #  | Dimension                   | <span class="rating rating-good">Good | <span class="rating rating-moderate">Moderate | <span class="rating rating-high-risk">High Risk | Measured                                                                                                                                                                                                                                                             | Rating                                          |
|----|-----------------------------|---------------------------------------|-----------------------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| D1 | Code Repository Health      | all checks pass                       | 1–2 gaps                                      | 3+ gaps / no CI                                 | <code>.gitignore</code> present, 4 CI workflows, lock files committed; but no <code>CODEOWNERS</code> , no branch protection signals, no PR template                                                                                                                 | <span class="rating rating-moderate">Moderate   |
| D2 | Third-Party Tool Usage      | mostly wired & current                | some unused/unwired                           | many unused/unmaintained                        | 15 of 19 production deps actively wired; 4 declared but not evidently used in app code (Pusher, Mailgun, AWS SDK vars, <code>react-router-dom</code> )                                                                                                               | <span class="rating rating-moderate">Moderate   |
| D3 | AI Tool / Agentic Readiness | ready                                 | partial                                       | not ready                                       | No <code>.cursor/</code> , <code>CLAUDE.md</code> , <code>.kiro/</code> , or Copilot config; committed secrets in <code>config/ivr_legacy.php</code> block clean checkout for automation; IVR modules are structurally enumerable but not CI-verifiable in isolation | <span class="rating rating-high-risk">High Risk |
| D4 | Database Usage              | sound                                 | some gaps                                     | no constraints / shared flat schema             | IVR dashboard tables have FK constraints; CRM core tables use integer columns with indexes but no FK constraints; 46 legacy IVR tables use identical JSON-blob schema with no relational constraints                                                                 | <span class="rating rating-moderate">Moderate   |
| D5 | Development Environment     | reproducible                          | partial                                       | manual / fragile                                | <code>.env.example</code> present; README has clear steps; but no Docker/devcontainer; no pre-commit hooks; linters configured but not enforced locally; PHPStan at level 1/9                                                                                        | <span class="rating rating-high-risk">High Risk |

8.8 Actions Required

| Gap                                                            | Action                                                                                                      | Rating                                          | Priority                                |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------|
| Hard-coded secrets in <code>config/ivr_legacy.php</code>       | Move to <code>.env</code> vars; purge from Git history with BFG                                             | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| No branch protection or code review signals                    | Add <code>CODEOWNERS</code> , PR template, enable branch protection rules                                   | <span class="rating rating-moderate">Moderate   | <span class="sev sev-high">High         |
| No containerized dev environment                               | Generate <code>docker-compose.yml</code> via Sail or manual Dockerfile                                      | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High         |
| Code style not enforced locally                                | Install <code>husky</code> + <code>lint-staged</code> ; raise PHPStan to level 5+                           | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High         |
| No AI-assisted tooling config                                  | Add <code>CLAUDE.md</code> , <code>.cursor/rules</code> , module manifest JSON                              | <span class="rating rating-high-risk">High Risk | <span class="sev sev-medium">Medium     |
| Missing FK constraints on CRM tables                           | Add migration with <code>foreignId()-&gt;constrained()</code> for <code>account_id / organization_id</code> | <span class="rating rating-moderate">Moderate   | <span class="sev sev-medium">Medium     |
| <code>fakerphp/faker</code> in production <code>require</code> | Move to <code>require-dev</code>                                                                            | <span class="rating rating-moderate">Moderate   | <span class="sev sev-low">Low           |
| Minimal test coverage                                          | Add per-module PHPUnit + Vitest tests                                                                       | <span class="rating rating-moderate">Moderate   | <span class="sev sev-high">High         |
| <code>react-router-dom</code> v5 likely unused                 | Verify and remove if unused                                                                                 | <span class="rating rating-moderate">Moderate   | <span class="sev sev-low">Low           |
| 46-table migration in single file                              | Split into per-module or per-domain migrations                                                              | <span class="rating rating-moderate">Moderate   | <span class="sev sev-low">Low           |

Full report saved to `docs/discovery/08-technical-debt.md` with all evidence sections (8.1–8.5), three Mermaid diagrams, and expected outcomes.

```

{"stop_reason":"end_turn","session_id":"27abc41b-5ee6-4442-9d6e-a675c6b2a40c","total_cost_usd":"2.1733624999999996","usage":{"input_tokens":23,"cache_creation_input_tokens":89806,"cache_read_input_tokens":1372989,"output_tokens":23297,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":89806,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":{"input_tokens":1,"output_tokens":1933,"cache_read_input_tokens":92222,"cache_creation_input_tokens":6939,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":6939},"type":"message"},"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":6178,"outputTokens":18,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.006268,"contextWindow":200000,"maxOutputTokens":32000},"claude-opus-4-6":{"inputTokens":23,"outputTokens":23297,"cacheReadInputTokens":1372989,"cacheCreationInputTokens":89806,"webSearchRequests":0,"costUSD":2.1670944999999993,"contextWindow":200000,"maxOutputTokens":32000},"terminal_reason":"completed","fast_mode_state":"off","uuid":"abd3a9c0-eb73-4c15-a79b-e0c1d1f8fd38"}

```